

The University Of Lahore



Computer Lab Manual

For

Operating Systems

(CS-3534)

Course Instructor

Mr. Murtaza Najaf,
Ms. Tayyaba Fatima

Lab Instructor

Mr. Murtaza, Ms. Tayyaba, Mr.
Hassnain Ayoub

Semester

Fall 2025

Department of Computer Science
The University of Lahore, Lahore, Pakistan

Contents

LAB EXPERIMENT 1	9
Introduction to UNIX/ Linux Environment & Vi Editor	9
AIM.....	9
OBJECTIVES	9
EXPECTED OUTCOMES	9
Hardware & Software Requirements	9
Hardware.....	9
Software	10
THEORY	10
INTRODUCTION TO LINUX.....	10
Linux File System Structure	10
Important Commands.....	10
Practical Importance	11
Measuring Performance	11
PROCEDURE.....	11
CONCLUSION.....	12
VIVA QUESTIONS	12
LAB EXPERIMENT 2	14
Linux Basics, Commands, Compiling and Debugging C Programs	14
AIM.....	14
OBJECTIVES	14
EXPECTED OUTCOMES	14
HARDWARE & SOFTWARE REQUIREMENTS	14
Hardware.....	14
Software	14
THEORY	15
Linux Command Structure.....	15
BASIC LINUX COMMANDS COVERED	15
COMPILING C PROGRAMS USING GCC	16
DEBUGGING WITH GDB	16

USING MAKE UTILITY	16
PRACTICAL IMPORTANCE	17
MEASURING SATISFACTION	17
PROCEDURE.....	17
Exercise 1: Basic Linux Commands	17
Exercise 2: Write and Compile a C Program	18
Program 1: Debugging Example.....	18
Exercise 2: Debug this Factorial Program	19
EXERCISES FOR LAB	19
CONCLUSION.....	19
VIVA QUESTIONS	19
LAB Experiment 3.....	20
Process Creation with Fork, Exec and Wait	20
AIM.....	20
OBJECTIVES	20
EXPECTED OUTCOMES	20
HARDWARE & SOFTWARE REQUIREMENTS	20
Hardware.....	20
Software	21
THEORY	21
fork() System Call.....	21
exec() System Call Family	21
wait() System Call.....	21
Zombie Process.....	22
Orphan Process	22
PRACTICAL IMPORTANCE	22
MEASURING PERFORMANCE	22
PROCEDURE.....	22
Exercise 1.....	22
Simple Child Process Creation Using fork().....	22
Exercise 2.....	23
Implementing wait().....	23
Exercise 3.....	23

Parent Waits for Child Before Termination	23
Exercise 4.....	24
Create a Chain of n Processes	24
Exercise 5.....	24
Create a Fan of n Processes.....	24
Exercise 6.....	24
Execute ls -l in Child Using exec()	24
EXERCISES FOR LAB	25
CONCLUSION.....	25
VIVA QUESTIONS	25
LAB EXPERIMENT 4 & 5	26
Links and File I/O	26
AIM.....	26
OBJECTIVES	26
LEARNING OUTCOMES	26
HARDWARE & SOFTWARE REQUIREMENTS	26
Hardware.....	26
Software	26
THEORY	27
Hard Link	27
Symbolic (Soft) Link	27
File I/O in C	27
System Calls.....	27
PRACTICAL IMPORTANCE	27
EXPERIMENT DETAILS.....	28
Exercise 1: Create Hard & Soft Links	28
Commands	28
Exercise 2: Count Blanks, Characters & Lines.....	28
C Program	28
Exercise 3: Implement mv Using System Calls.....	29
Exercise 4: Implement ls Using System Calls	29
MEASURING SATISFACTION	30
PROCEDURE.....	30

CONCLUSION.....	30
VIVA QUESTIONS	31
LAB EXPERIMENT 6 & 7	32
Pipes and Redirection	32
AIM.....	32
OBJECTIVES	32
LEARNING OUTCOMES	32
HARDWARE & SOFTWARE REQUIRED.....	33
Hardware.....	33
Software	33
THEORY	33
Pipes.....	33
Properties	33
System Call	33
Redirection.....	33
dup2().....	34
fork()	34
execl()	34
EXERCISES	34
Exercise 1: Parent writes to a pipe, child reads.....	34
Exercise 2: Implement <code>ls -l sort -n +4</code>	35
Exercise 3: Redirect STDOUT to file and append later.....	36
Exercise 4: Implement	36
Exercise 5: Redirection Practice using cat.....	36
PRACTICAL IMPORTANCE	37
MEASURING SATISFACTION	37
PROCEDURE.....	37
CONCLUSION.....	38
VIVA QUESTIONS	38
LAB EXPERIMENT 8 , 9 & 10	39
Signals & Threads.....	39
AIM.....	39
OBJECTIVE	39

LEARNING OUTCOMES	39
Hardware Requirements.....	39
Software Requirements.....	39
Theory	39
Exercise 1: Blocking and Unblocking SIGINT	40
Exercise 2: Two Threads Printing A and B	40
Procedure.....	41
Exercise 3: Matrix Multiplication using Threads and Condition Variables.....	41
Exercise 4: Asynchronous File Copy using Signals	42
Practical Importance	44
Measuring Satisfaction.....	44
Conclusion	45
Viva Questions.....	45
LAB EXPERIMENT 11	46
System-V inter process communication mechanisms. Semaphore array.....	46
AIM.....	46
OBJECTIVES	46
LEARNING OUTCOMES	46
HARDWARE REQUIREMENTS.....	46
SOFTWARE REQUIREMENTS	46
THEORY	46
PROCEDURE.....	47
Exercise 2 – Semaphore Creation Function.....	49
PRACTICAL IMPORTANCE	49
MEASURING SATISFACTION	49
CONCLUSION.....	50
VIVA QUESTIONS	50
LAB EXPERIMENT 13-14.....	51
Connection Oriented Communication, Little and Big Endian	51
AIM.....	51
OBJECTIVES	51
LEARNING OUTCOMES	51
HARDWARE REQUIREMENTS.....	51

SOFTWARE REQUIREMENTS	51
THEORY	51
PROCEDURE.....	54
PRACTICAL IMPORTANCE	55
MEASURING SATISFACTION	55
CONCLUSION.....	55
VIVA QUESTIONS	55
LAB EXPERIMENT 15	56
Advanced Client–Server Programs.....	56
AIM.....	56
OBJECTIVES	56
LEARNING OUTCOMES	56
THEORY	56
Exercise 2: Character Increment Programs.....	56
Server Logic:.....	56
Client Logic:	56
Exercise 3: Integer Increment Programs	57
Server Logic:.....	57
Client Logic:	57
Exercise 4: Multi-threaded Server	57
Server Features:	57
PROCEDURE.....	57
PRACTICAL IMPORTANCE	57
MEASURING SATISFACTION	57
CONCLUSION.....	58
VIVA QUESTIONS	58
Lab Experiment 16 & 17.....	59
Messages and Queues	59
AIM.....	59
OBJECTIVES	59
LEARNING OUTCOMES	59
HARDWARE REQUIREMENTS.....	59
SOFTWARE REQUIREMENTS	59

THEORY 59

PROCEDURE..... 60

PRACTICAL IMPORTANCE 60

MEASURING SATISFACTION 60

CONCLUSION..... 60

VIVA QUESTIONS 61

LAB EXPERIMENT 1

Introduction to UNIX/ Linux Environment & Vi Editor

AIM

The aim of this lab is to introduce students to the Linux operating system, its file system structure, and basic command-line operations used for file handling, navigation, and file compression.

OBJECTIVES

By the end of this lab, students will be able to:

1. Understand the history and philosophy of Linux and differentiate it from Windows.
2. Install Ubuntu Linux using VirtualBox or through dual boot configuration.
3. Navigate the Linux directory structure using basic commands.
4. Create, delete, copy, and manage files and directories using the terminal.
5. Search text inside files using commands like `grep` and `find`.
6. Compress and extract files using `tar` command.
7. Develop confidence in using Linux as a development environment.

EXPECTED OUTCOMES

After completing this lab, students will be able to:

- Use Linux terminal commands with confidence.
- Effectively manage files and directories.
- Understand the Linux filesystem hierarchy.
- Perform system navigation and search tasks.
- Compress and extract files in Linux.
- Differentiate the Linux environment from Windows operational behavior.

Hardware & Software Requirements

Hardware

- the A personal computer or laptop with:
 - Minimum 4 GB RAM (8 GB recommended)
 - 25 GB free storage space
 - Processor with virtualization support

Software

- **Ubuntu Linux** (pre-installed or installed via VirtualBox)
- **Oracle VirtualBox** (if not using dual boot)
- Linux terminal (default in Ubuntu)

THEORY

INTRODUCTION TO LINUX

Linux is an open-source operating system based on UNIX. It offers powerful command-line tools, a hierarchical file system, and is widely used in servers, cloud infrastructure, cybersecurity, networking, and programming environments.

Linux vs Windows (Key Differences)

Feature	Linux	Windows
File System	Hierarchical, case-sensitive	Not case-sensitive
Access	Open source	Proprietary
Interface	Command Line + GUI	Mostly GUI
Security	High (permission-based)	Moderate
Package Management	APT, Yum	Windows Installer

Linux File System Structure

Linux follows a tree-like directory structure:

- `/` – Root
- `/home` – User files
- `/etc` – Configuration files
- `/bin` – Essential commands
- `/var` – Logs, variable files

Important Commands

- `pwd` – Show current directory
- `ls` – List directory contents
- `cd` – Change directory
- `mkdir, rmdir` – Create/remove directory
- `cp, mv, rm` – Copy/move/delete files

- `cat, nano` – View/Edit files
- `grep, find` – Search content/files
- `tar -cvf` – Compress
- `tar -xvf` – Extract

Practical Importance

- Linux is the backbone of servers, cloud computing, networking devices, and cybersecurity setups.
- Most programming environments (Python, C, AI/ML frameworks) run best on Linux.
- Learning Linux helps in DevOps, ethical hacking, system administration, and research computing.
- Students gain the ability to automate tasks, manage systems, and work on real development platforms used in industry.

Measuring Performance

The performance can be evaluated through:

1. **Observation:** Students must run all commands independently.
2. **Task Completion:**
 - Create and manage directories/files
 - Search content inside files
 - Compress and extract files
3. **Quiz/Viva:** Assess understanding of filesystem and commands.
4. **Submission:** Students should submit screenshots or command logs.
5. **Feedback Form:** Evaluate clarity of instructions and lab usefulness.

PROCEDURE

Step 1: Install Ubuntu (if not preinstalled)

- Install via VirtualBox or dual boot.
- Configure username and password.

Step 2: Launch the Terminal

Open the terminal using:

`Ctrl + Alt + T`

Step 3: Navigate the File System

Run:

`pwd`

```
ls
cd /home
ls -l
```

Step 4: Create Directories and Files

```
mkdir lab1
cd lab1
touch file1.txt
nano file1.txt
```

Step 5: Copy, Delete, and Move Files

```
cp file1.txt file2.txt
mv file2.txt backup.txt
rm backup.txt
```

Step 6: Searching in Files

```
grep "hello" file1.txt
find /home -name "*.txt"
```

Step 7: Compress and Uncompress Files

```
tar -cvf lab1_backup.tar lab1
tar -xvf lab1_backup.tar
```

CONCLUSION

In this lab, students were introduced to the Linux environment and basic command-line operations. They learned about Linux installation, directory structure, file handling, searching tools, and compression utilities. This foundation will help them perform advanced tasks in future labs and programming environments.

VIVA QUESTIONS

1. What is Linux? How is it different from Windows?
2. What does the command `pwd` do?
3. Explain the Linux file system hierarchy.
4. What is the difference between `rm` and `rmdir`?
5. How do you create multiple directories at once?
6. What is the purpose of the `grep` command?
7. Write the command to compress a folder using `tar`.
8. How do you extract a tar file?
9. What is the purpose of the `/home` directory?
10. Why is Linux preferred for development and servers?

LAB EXPERIMENT 2

Linux Basics, Commands, Compiling and Debugging C Programs

AIM

To introduce Linux shell commands, GCC compilation techniques, Makefile usage, and debugging C programs using the GDB debugger.

OBJECTIVES

By the end of this experiment, students will:

1. Learn fundamental Linux commands for navigating the filesystem.
2. Use the Linux command-line interface (CLI) efficiently.
3. Write, compile, and execute C programs using GCC.
4. Understand how to compile multi-file programs using the **make** utility.
5. Debug C programs using the **gdb** debugger.
6. Identify logical and runtime errors in C programs with debugging tools.

EXPECTED OUTCOMES

After completing this lab, students will be able to:

- Execute essential Linux commands such as `mkdir`, `ls`, `cd`, `rm`, `pwd`, `cp`, `mv`, `gcc`.
- Write and compile C programs using GCC with appropriate flags.
- Apply the **-Wall** and **-g** options to improve debugging and code quality.
- Use **gdb** to set breakpoints, inspect variables, and trace execution.
- Correct errors in faulty C programs using debugging techniques.
- Understand the workflow of compiling, linking, and executing programs in Linux.

HARDWARE & SOFTWARE REQUIREMENTS

Hardware

- PC or laptop with at least 4 GB RAM
- Virtualization support enabled (for VirtualBox users)

Software

- Ubuntu Linux (preinstalled or via VirtualBox)

- GCC Compiler
- GDB Debugger
- Text Editor (gedit, nano, vim, or VS Code)
- make utility

THEORY

Linux Command Structure

A typical Linux command follows the format:

```
command [options] [arguments]
```

Example:

```
ls -l data.txt
```

- `ls` → command
- `-l` → option
- `data.txt` → argument

Options typically start with - and multiple options can be combined.

BASIC LINUX COMMANDS COVERED

Command	Purpose
<code>Pwd</code>	Print working directory
<code>Ls</code>	List files
<code>Cd</code>	Change directory
<code>Mkdir</code>	Create directory
<code>Rm</code>	Remove file
<code>Cp</code>	Copy file
<code>Mv</code>	Move/rename file
<code>Man</code>	Open manual page
<code>Gcc</code>	Compile C programs

COMPILING C PROGRAMS USING GCC

GCC basic syntax:

```
gcc [options] file.c -o outputname
```

Common flags:

- `-Wall` → show warnings
- `-g` → include debug information

DEBUGGING WITH GDB

Key gdb commands:

Command	Description
<code>l</code> or <code>list</code>	Show code
<code>b N</code>	Set breakpoint at line N
<code>Run</code>	Run the program
<code>N</code>	Execute next line
<code>p var</code>	Print variable value
<code>C</code>	Continue execution
<code>Q</code>	Quit gdb

USING MAKE UTILITY

A Makefile simplifies multi-file compilation.

Basic example:

```
all:
    gcc main.c fun.c -o app
```

Run using:

make

PRACTICAL IMPORTANCE

- Linux is widely used in industry, cloud, servers, AI research, and cybersecurity.
- Debugging skills are essential for software development.
- Compilation and linking knowledge is required for system programming.
- Makefiles automate builds for large projects.
- GDB is a professional debugging tool used globally.

MEASURING SATISFACTION

1. **Execution accuracy** – the student must run commands correctly.
2. **Compilation success** – must compile given programs.
3. **Debugging skill** – ability to identify errors using `gdb`.
4. **Completion of exercises** – students must attempt all tasks.
5. **Viva questions** – to assess conceptual understanding.
6. **Lab report submission** – screenshots or command outputs.

PROCEDURE

Exercise 1: Basic Linux Commands

Run the following commands:

```
pwd
ls
mkdir test
cd test
touch file1.txt
cp file1.txt copy.txt
mv copy.txt renamed.txt
rm renamed.txt
man mkdir
```

Explore manual pages:

```
man ls
man rm
man cp
```

Exercise 2: Write and Compile a C Program

Create a C program:

```
gedit hello.c
```

Add:

```
#include <stdio.h>
int main() {
    printf("Hello, world!\n");
    return 0;
}
```

Compile:

```
gcc -Wall hello.c -o hello
```

Run:

```
./hello
```

Program 1: Debugging Example

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char argv) {
    char *buf;
    buf = malloc(1<<31);
    fgets(buf, 1024, stdin);
    printf("%s\n", buf);
    return 1;
}
```

Compile with debug info:

```
gcc -g segfault.c -o seg
```

Run gdb:

```
gdb seg
(1)
(b 8)
(run)
(p buf)
(n)
```

Exercise 2: Debug this Factorial Program

```
#include <stdio.h>
int main() {
    int i, num, j;
    printf("Enter the number: ");
    scanf("%d", &num);
    for (i=1; i<num; i++)
        j=j*i;
    printf("The factorial of %d is %d\n", num, j);
}
```

Students must use:

```
gcc -g factorial.c -o fact
gdb fact
```

EXERCISES FOR LAB

3. Write a C Program to Find the Largest Among 3 Numbers.
4. Write a C Program to Check Whether a Number is Even or Odd.
5. Use `gdb` debugger to debug programs using `-g` flag.

CONCLUSION

In this experiment, students learned fundamental Linux commands, C program compilation using GCC, Makefile usage, and debugging with `gdb`. These skills are essential for systems programming, embedded systems, and software development. The students now understand error tracing, breakpoints, and command-line-based software development techniques.

VIVA QUESTIONS

1. What is the general structure of a Linux command?
2. Why is GCC commonly used on Linux?
3. What is the purpose of the `-Wall` option in GCC?
4. What does the `-g` flag do?
5. What is a segmentation fault?
6. How do you set a breakpoint in `gdb`?
7. What is the difference between `next` and `step` in `gdb`?
8. What is a Makefile and why is it useful?
9. How do you view the manual page of a command in Linux?
10. What is the difference between compile-time and runtime errors?

LAB EXPERIMENT 3

Process Creation with Fork, Exec and Wait

AIM

To understand how processes are created, replaced, and synchronized in Linux using the system calls **fork()**, **exec()**, and **wait()**.

OBJECTIVES

By the end of this lab, students will:

1. Learn how processes are created using **fork()**.
2. Understand how a running process can be replaced using **exec()**.
3. Use **wait()** to synchronize parent and child processes.
4. Identify and explain **zombie** and **orphan** processes.
5. Write, compile, and execute C programs demonstrating process creation and management.
6. Create multiple processes (chain and fan) using loops and command-line arguments.

EXPECTED OUTCOMES

Students will be able to:

- Demonstrate creation of child processes using **fork()**
- Use **exec()** to execute external programs in child processes
- Use **wait()** to prevent zombie processes
- Understand parent–child relationships (PID, PPID)
- Write programs that create chains and fans of processes
- Debug basic process-related issues

HARDWARE & SOFTWARE REQUIREMENTS

Hardware

- Standard PC or laptop (4–8 GB RAM)

Software

- Ubuntu Linux (native or via VirtualBox)
 - GCC Compiler
 - Text editor (gedit, nano, vim, VSCode)
-

THEORY

fork() System Call

`fork()` creates a new process (child).

It returns:

- **0** → in child process
- **positive PID** → in parent
- **-1** → on failure

Child receives a copy of the parent's memory image.

exec() System Call Family

`exec()` replaces the current process image with a new program.

Examples:

- `execl()`
- `execv()`
- `execvp()`

After `exec`, the old program **never continues** unless `exec` fails.

wait() System Call

Causes the parent to **wait** until its child completes execution.

- Prevents **zombie processes**
- Returns the **PID** of terminated child
- Stores termination status

Zombie Process

A process that has terminated but still has an entry in the process table because its parent did not call `wait()`.

Orphan Process

A process whose parent has terminated before it.
The orphan is adopted by **init (PID 1)**.

PRACTICAL IMPORTANCE

- Used in **OS kernels, shells, and servers**
- Background jobs and multitasking rely on `fork/exec`
- Process creation and synchronization are essential for system programming
- Tools like Apache, MySQL, SSH use `fork()`
- `exec` is used to run commands from shells

MEASURING PERFORMANCE

Evaluation is based on:

- Correct execution of programs
- Understanding parent/child output
- Ability to explain zombie/orphan
- Completion of chain/fan process programs
- Viva questions
- Lab report formatting and screenshots

PROCEDURE

Exercise 1

Simple Child Process Creation Using `fork()`

Source Code

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("original process with pid %d ppid %d\n", getpid(), getppid());
    pid_t pid = fork();

    if (pid != 0)
        printf("parent process with pid %d ppid %d\n", getpid(), getppid());
```

```

    else {
        sleep(5);
        printf("child process with pid %d ppid %d\n", getpid(), getppid());
    }

    printf("pid %d terminates\n", getpid());
    return 0;
}

```

Exercise 2

Implementing wait()

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

int main() {
    int pid, status;

    pid = fork();
    if (pid == -1) {
        printf("fork failed\n");
        exit(1);
    }

    if (pid == 0) {    // Child
        printf("Child here!\n");
    }
    else {             // Parent
        wait(&status);
        printf("Well done kid!\n");
    }
}

```

Exercise 3

Parent Waits for Child Before Termination

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

int main() {
    int pid, status;
    printf("Hello World!\n");

    pid = fork();
    if (pid == -1) {
        perror("bad fork");
        exit(1);
    }
}

```

```

    }

    if (pid == 0)
        printf("I am the child process.\n");

    else {
        wait(&status);
        printf("I am the parent process.\n");
    }
}

```

Exercise 4

Create a Chain of n Processes

(Each process creates exactly one child)

Example structure:

P1 → P2 → P3 → ... → Pn

Exercise 5

Create a Fan of n Processes

(Parent creates n children simultaneously)

Example structure:

```

    P
   /\
  C1 C2 ... Cn

```

Exercise 6

Execute ls -l in Child Using exec()

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t childpid = fork();

    if (childpid == 0) {
        execl("/bin/ls", "ls", "-l", NULL);
    }

    wait(NULL);
}

```



```
    return 0;  
}
```

EXERCISES FOR LAB

Exercise 4 – Chain of n processes

Exercise 5 – Fan of n processes

CONCLUSION

In this lab, students learned how to create processes with `fork()`, replace them using `exec()`, and synchronize using `wait()`. They also explored zombies, orphans, fan creation, and process chains. These concepts form the foundation of process management in Unix/Linux operating systems.

VIVA QUESTIONS

1. What does `fork()` return?
2. What is the difference between parent and child process?
3. What is a zombie process?
4. What is an orphan process?
5. Why do we use `wait()`?
6. What is the purpose of `exec()`?
7. What is the difference between `execl()` and `execvp()`?
8. Can `exec()` return? Why or why not?
9. What happens if you do not call `wait()`?
10. Can a child become a parent? Explain.

LAB EXPERIMENT 4 & 5

Links and File I/O

AIM

To understand Linux file linking mechanisms (Hard & Soft links) and perform File Input/Output operations (ASCII & Binary) using C programming.

OBJECTIVES

After completing this lab, the student will be able to:

- Differentiate **Hard Links** and **Symbolic Links** in Linux.
- Create and manage links using shell commands.
- Read and write files using **standard I/O** and **system calls**.
- Handle character-level file processing (ASCII and binary).
- Understand file metadata such as **inode numbers**.

LEARNING OUTCOMES

Students will learn to:

- Implement file handling using `fopen`, `fgetc`, `fread`, `write`, `open`, `unlink`.
- Create shell-level links using `ln` and `ln -s`.
- Write programs that perform **file copying**, **counting**, and **directory listing** using system calls.
- Demonstrate practical understanding of Linux file system behavior.

HARDWARE & SOFTWARE REQUIREMENTS

Hardware

- Any computer capable of running Linux

Software

- Linux OS (Ubuntu recommended)
- GCC Compiler
- Terminal / Shell

- Text Editor (Nano, Vim, VS Code)

THEORY

Hard Link

- A hard link is another name for an existing file.
- Hard links share the **same inode number**.
- If original file is deleted, the hard link remains functional.
- Cannot link directories or cross different partitions.

Symbolic (Soft) Link

- A symbolic link is a pointer to the file path.
- Has a **different inode number**.
- If original file is deleted, the symlink becomes **broken**.
- Can link directories and cross filesystems.

File I/O in C

Two types:

- **Text I/O** → Using `fgetc`, `fputc`, `fgets`
- **Binary I/O** → Using `fread`, `fwrite`

System Calls

Common system calls:

- `open()`
- `read()`
- `write()`
- `close()`
- `unlink()`
- `opendir()`
- `readdir()`

PRACTICAL IMPORTANCE

- File systems are fundamental in OS design.
- Linking allows efficient storage and fast access.
- File I/O is the backbone of:

- Compilers
 - Databases
 - Operating systems
 - Embedded systems
- System calls help understand how OS interacts with hardware.

EXPERIMENT DETAILS

Exercise 1: Create Hard & Soft Links

Commands

```
touch myfile
echo "Hello OS" > myfile
ln myfile link1
ln myfile link2
ls -il
ln -s myfile myfile_symlink
ls -il
```

Exercise 2: Count Blanks, Characters & Lines

C Program

```
#include <stdio.h>

int main() {
    FILE *fp;
    int blanks = 0, lines = 0, chars = 0;
    char ch;

    fp = fopen("text.txt", "r");
    if (fp == NULL) {
        printf("File not found!\n");
        return 1;
    }

    while ((ch = fgetc(fp)) != EOF) {
        if (ch == ' ')
            blanks++;
        if (ch == '\n')
            lines++;
        chars++;
    }

    fclose(fp);

    printf("No. of blanks: %d\n", blanks);
    printf("No. of lines: %d\n", lines);
    printf("No. of characters: %d\n", chars);
}
```

```

    return 0;
}

```

Exercise 3: Implement mv Using System Calls

```

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    int fd1, fd2;
    char buffer[1024];
    int n;

    fd1 = open("old.txt", O_RDONLY);
    if (fd1 < 0) {
        perror("Open old file");
        return 1;
    }

    fd2 = open("new.txt", O_WRONLY | O_CREAT, 0644);

    while ((n = read(fd1, buffer, sizeof(buffer))) > 0) {
        write(fd2, buffer, n);
    }

    close(fd1);
    close(fd2);

    unlink("old.txt"); // Delete original file

    printf("Move using system calls completed.\n");

    return 0;
}

```

Exercise 4: Implement ls Using System Calls

```

#include <stdio.h>
#include <dirent.h>

int main() {
    DIR *dp;
    struct dirent *entry;

    dp = opendir(".");
    if (dp == NULL) {
        perror("opendir");
        return 1;
    }

    while ((entry = readdir(dp)) != NULL) {

```

```
        printf("Name: %s\t Inode: %lu\n", entry->d_name, entry->d_ino);
    }

    closedir(dp);
    return 0;
}
```

MEASURING SATISFACTION

Students should be able to:

- Correctly explain concepts of links.
- Successfully execute all commands and programs.
- Demonstrate outputs as expected.
- Answer viva questions confidently.

Assessment includes:

- Execution correctness
- Code quality
- Understanding of inode, file linking, and I/O

PROCEDURE

1. Start terminal.
2. Create sample files.
3. Create hard and soft links.
4. Observe inode numbers.
5. Compile C programs using:
6. `gcc program.c -o output`
7. `./output`
8. Perform text and binary file I/O experiments.
9. Record outputs and conclusions.

CONCLUSION

In this lab, you learned the difference between **hard** and **symbolic links**, explored how files are linked within the Linux filesystem, and practiced different methods of file input/output. You also used important OS system calls such as `open`, `read`, `write`, `unlink`, and `readdir`. This experiment strengthens your practical understanding of how operating systems manage files internally.

VIVA QUESTIONS

1. What is an inode?
2. Difference between hard link and soft link?
3. Can a symbolic link survive if the original file is deleted?
4. Why can't hard links link directories?
5. What happens if two hard links are created for the same file?
6. What is the difference between `fgetc()` and `fread()`?
7. Why are system calls slower than standard I/O functions?
8. What does `unlink()` do?
9. Explain the output of `ls -il`.
10. What does `opendir()` return?

LAB EXPERIMENT 6 & 7

Pipes and Redirection

AIM

To understand and implement Inter-Process Communication (IPC) using **pipes**, and to learn how to use **input/output redirection** in Linux.

OBJECTIVES

By the end of this lab, students will be able to:

1. Write, compile, and execute C programs using:
 - o `pipe()`
 - o `fork()`
 - o `dup2()`
 - o `execl()`
2. Redirect input/output using:
 - o `>`
 - o `<`
 - o `>>`
 - o `|`
 - o `cat` command
3. Build simple pipelines like `ls -l | sort, tail | grep | sort, etc.`

LEARNING OUTCOMES

Students will learn to:

- Use pipes for communication between parent-child processes.
- Redirect standard input/output to files and processes.
- Understand how UNIX pipelines work internally.
- Use system calls to create complex command-line behaviors programmatically.

HARDWARE & SOFTWARE REQUIRED

Hardware

- A computer capable of running Linux.

Software

- Linux (Ubuntu preferred)
- GCC compiler
- Shell/Terminal (bash)
- Text Editor (nano/vim/VS Code)

THEORY

Pipes

A pipe is a **unidirectional communication channel** used between related processes (usually parent-child).

Properties

- Data flows **one way only** (write end → read end).
- Implemented using a **buffer in kernel memory** (approx. 10 KB).
- If the reader reads before data is written → it waits.
- If writer writes and buffer is full → it waits.

System Call

```
int fd[2];  
pipe(fd);
```

- `fd[0]` → read end
- `fd[1]` → write end

Redirection

Operators

Symbol	Meaning
>	Redirect STDOUT to file (overwrite)
>>	Append output to file

Symbol	Meaning
--------	---------

<	Redirect file as STDIN
---	------------------------

,

Example:

```
ls -l | more
```

dup2()

Redirects file descriptors:

```
dup2(fd[1], STDOUT_FILENO);
```

Now STDOUT points to pipe write end.

fork()

Creates a child process. Both processes share the pipe.

execl()

Replaces current program with a new executable.

EXERCISES

Exercise 1: Parent writes to a pipe, child reads

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#define BUFSIZE 10

int main(void) {
    char bufin[BUFSIZE] = "empty";
    char bufout[] = "hello";
    int bytesin;
    pid_t childpid;
    int fd[2];

    if (pipe(fd) == -1) {
        perror("Failed to create the pipe");
        return 1;
    }
}
```

```

bytesin = strlen(bufin);
childpid = fork();
if (childpid == -1) {
    perror("Failed to fork");
    return 1;
}
if (childpid) /* parent */
    write(fd[1], bufout, strlen(bufout)+1);
else /* child */
    bytesin = read(fd[0], bufin, BUFSIZE);

fprintf(stderr, "[%ld]:my bufin is {%.*s}, my bufout is {%s}\n",
        (long)getpid(), bytesin, bufin, bufout);
return 0;
}

```

Exercise 2: Implement `ls -l | sort -n +4`

```

#include <errno.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main(void) {
    pid_t childpid;
    int fd[2];

    if ((pipe(fd) == -1) || ((childpid = fork()) == -1)) {
        perror("Failed to setup pipeline");
        return 1;
    }

    if (childpid == 0) { /* child: ls */
        if (dup2(fd[1], STDOUT_FILENO) == -1)
            perror("Failed to redirect stdout of ls");
        else if ((close(fd[0]) == -1) || (close(fd[1]) == -1))
            perror("Failed to close extra pipe descriptors on ls");
        else {
            execl("/bin/ls", "ls", "-l", NULL);
            perror("Failed to exec ls");
        }
        return 1;
    }
    /* parent: sort */
    if (dup2(fd[0], STDIN_FILENO) == -1)
        perror("Failed to redirect stdin of sort");
    else if ((close(fd[0]) == -1) || (close(fd[1]) == -1))
        perror("Failed to close extra pipe file descriptors on sort");
    else {
        execl("/bin/sort", "sort", "-n", "+4", NULL);
        perror("Failed to exec sort");
    }
    return 1;
}

```

Exercise 3: Redirect STDOUT to file and append later

(You can implement using `dup2()` and `open()`.)
I can write the full program if needed.

Exercise 4: Implement

```
tail -5 alpha.txt | grep ee | sort
```

(using pipes + `fork()` for 3 commands)

Exercise 5: Redirection Practice using cat

Commands:

a)

```
echo "a b c" > tmp.txt
```

b)

```
cat tmp.txt
```

c)

```
cat > tmp.txt
1
2
3
Ctrl + D
```

d)

```
cat tmp.txt
```

e)

```
cat >> tmp.txt
a
b
c
```

Ctrl + D

f)

```
cat < tmp.txt
```

g)

```
cat < tmp.txt > tmp2.txt
```

h)

```
cat tmp2.txt
```

i)

```
ps > file1.txt
```

j)

```
ps | more
```

PRACTICAL IMPORTANCE

- Pipes allow processes to communicate efficiently.
- Nearly all UNIX commands internally use pipelines.
- Redirection is essential for automation, scripting, and system administration.
- Helps students understand how shells like **bash**, **zsh**, and **csh** execute pipelines.

MEASURING SATISFACTION

Students should be able to:

- Correctly create and use pipes in C.
- Redirect output/input using operators.
- Build pipelines using both shell and C system calls.
- Demonstrate working outputs for each exercise.
- Pass viva questions confidently.

PROCEDURE

1. Open terminal.
2. Create a new C file.
3. Implement pipe-based programs.

4. Compile:
5. `gcc pipe1.c -o pipe1`
6. `./pipe1`
7. Test redirection commands.
8. Observe how STDIN/STDOUT change.
9. Document results.

CONCLUSION

This lab provided practical experience with **inter-process communication** using pipes and demonstrated how UNIX performs **I/O redirection**. Students learned the behavior of `pipe()`, `dup2()`, and `fork()` system calls and implemented pipelines similar to Linux shell commands. Understanding pipes is essential for advanced OS development, shell implementation, and parallel processing.

VIVA QUESTIONS

1. What is a pipe?
2. Why is pipe unidirectional?
3. What is the difference between pipe and FIFO?
4. What does `dup2()` do?
5. What happens when pipe buffer is full?
6. How does `ls -l | sort` work internally?
7. Why do we close unused pipe ends?
8. Can two unrelated processes use `pipe()`?
9. What is STDIN, STDOUT, STDERR?
10. What is the purpose of file descriptors 0,1,2?

LAB EXPERIMENT 8 , 9 & 10

Signals & Threads

AIM

To understand and implement the working of signals and multithreading concepts using POSIX threads in C.

OBJECTIVE

1. To understand the concept of threads and thread IDs
2. To learn the use of POSIX thread library functions
3. To understand signal handling mechanisms in Linux
4. To implement multithreaded programs
5. To understand asynchronous I/O using signals

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Differentiate between process ID and thread ID
2. Create and manage multiple threads
3. Use thread synchronization techniques
4. Handle signals in Linux
5. Implement asynchronous file operations using signals

Hardware Requirements

- PC / Laptop
- Minimum 2 GB RAM

Software Requirements

- Linux Operating System
- GCC Compiler
- POSIX Threads library (pthread)

Theory

A **thread** is the smallest unit of execution within a process. Every process has at least one thread called the main thread.

Key differences between Process ID and Thread ID:

- Process ID is unique system-wide, while thread ID is unique within a process.
- Process ID is usually an integer, but thread ID may be a structure.
- Process ID is easy to print, thread ID is not.

Important thread functions:

- `pthread_create()` → creates a new thread
- `pthread_self()` → returns the calling thread's ID
- `pthread_equal()` → compares two thread IDs

A **signal** is a software interrupt used by the operating system to notify a process that a specific event has occurred.

Asynchronous I/O allows file operations to occur while the program performs other tasks, improving efficiency.

Exercise 1: Blocking and Unblocking SIGINT

To write a program that blocks and unblocks the SIGINT signal.

Procedure

1. Write a C program to block SIGINT.
2. Wait for a few seconds.
3. Unblock SIGINT.
4. Compile and run the program.

Sample Code

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int main() {
    sigset_t newset;

    sigemptyset(&newset);
    sigaddset(&newset, SIGINT);

    printf("Blocking SIGINT...\n");
    sigprocmask(SIG_BLOCK, &newset, NULL);

    sleep(5);

    printf("Unblocking SIGINT...\n");
    sigprocmask(SIG_UNBLOCK, &newset, NULL);

    while(1);
    return 0;
}
```

Exercise 2: Two Threads Printing A and B

To create two threads where one prints 'A' and the other prints 'B'.

Procedure

- Create two threads using pthread_create().
- One thread prints A.
- The other thread prints B.

Sample Code

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
void* printA(void* arg) {  
    while(1) {  
        printf("A");  
    }  
    return NULL;  
}
```

```
void* printB(void* arg) {  
    while(1) {  
        printf("B");  
    }  
    return NULL;  
}
```

```
int main() {  
    pthread_t t1, t2;  
  
    pthread_create(&t1, NULL, printA, NULL);  
    pthread_create(&t2, NULL, printB, NULL);  
  
    pthread_join(t1, NULL);  
    pthread_join(t2, NULL);  
  
    return 0;  
}
```

Exercise 3: Matrix Multiplication using Threads and Condition Variables

To multiply a 3×3 matrix with itself using three threads and synchronize using condition variables.

Procedure

1. Create 3 threads, each handling one row.
2. Use mutex and condition variable to synchronize.
3. Print the resultant matrix after all threads finish.

Sample Logic

- Each thread computes one row.
- A counter tracks completed threads.
- When all threads finish, main thread prints the result.

Exercise 4: Asynchronous File Copy using Signals

To copy a file asynchronously using signals while doing other work.

Theory

This program:

- Uses signals to notify when I/O is completed.
- Allows the CPU to execute other work simultaneously.

Source Code

```
#include <errno.h>
#include <fcntl.h>
#include <signal.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>
#include "asyncmonitorsignal.h"

#define BLKSIZE 1024
#define MODE (S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH)

void dowork(void);

int main(int argc, char *argv[]) {
    char buf[BLKSIZE];
    int done = 0;
    int error;
    int fd1;
    int fd2;

    if (argc != 3) {
        fprintf(stderr, "Usage: %s filename1 filename2\n", argv[0]);
        return 1;
    }

    if ((fd1 = open(argv[1], O_RDONLY)) == -1) {
        fprintf(stderr, "Failed to open %s:%s\n", argv[1], strerror(errno));
        return 1;
    }

    if ((fd2 = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, MODE)) == -1) {
```

```

    fprintf(stderr, "Failed to open %s: %s\n", argv[2], strerror(errno));
    return 1;
}

if (initsignal(SIGRTMAX) == -1) {
    perror("Failed to initialize signal");
    return 1;
}

if (initread(fd1, fd2, SIGRTMAX, buf, BLKSIZE) == -1) {
    perror("Failed to initiate the first read");
    return 1;
}

for (;;) {
    dowork();
    if (!done)
        if (done = getdone())
            if (error = geterror())
                fprintf(stderr, "Failed to copy file:%s\n", strerror(error));
            else
                fprintf(stderr, "Copy successful, %d bytes\n", getbytes());
}
}

```

Source Code

```

#include<stdio.h>
#include<string.h>
#include<pthread.h>
#include<stdlib.h>
#include<unistd.h>

```

```

pthread_t tid[2];

```

```

void* doSomething(void *arg)
{
    unsigned long i = 0;
    pthread_t id = pthread_self();

    if(pthread_equal(id,tid[0]))
    {
        printf("\n First thread processing\n");
    }
    else
    {
        printf("\n Second thread processing\n");
    }
}

```

```

    for(i=0; i<(0xFFFFFFFF);i++);

    return NULL;
}

int main(void)
{
    int i = 0;
    int err;

    while(i < 2)
    {
        err = pthread_create(&(tid[i]), NULL, &doSomething, NULL);
        if (err != 0)
            printf("\ncan't create thread :[%s]", strerror(err));
        else
            printf("\n Thread created successfully\n");

        i++;
    }

    sleep(5);
    return 0;
}

```

Practical Importance

- Used in real-time systems
- Important in operating system design
- Improves application performance using parallel execution
- Used heavily in servers and multitasking environments

Measuring Satisfaction

Students will be evaluated based on:

- Correct creation of threads
- Proper use of pthread_create()
- Correct usage of signals
- Proper handling of SIGINT blocking/unblocking
- Correct asynchronous file copying behavior
- Proper synchronization using condition variables
- Correct program output
- Understanding during viva

Conclusion

This experiment demonstrated the use of threads and signals in C programming. The use of multithreading improved performance and enabled concurrent execution, while signals allowed controlled communication with processes.

Viva Questions

1. What is a thread?
2. Difference between process and thread?
3. What does `pthread_self()` do?
4. What is a signal?
5. What is SIGINT?
6. Why are condition variables used?
7. What is asynchronous I/O?
8. Difference between `pthread_join()` and condition variables?

LAB EXPERIMENT 11

System-V inter process communication mechanisms. Semaphore array

AIM

To understand and implement the Producer–Consumer problem using semaphores for process synchronization.

OBJECTIVES

1. To understand System V Inter-Process Communication (IPC) mechanisms
2. To understand the working of semaphores
3. To implement the Producer–Consumer problem
4. To learn mutual exclusion and synchronization

LEARNING OUTCOMES

After completing this lab, students will be able to:

1. Explain how semaphores work
2. Implement process synchronization using semaphores
3. Solve classic synchronization problems like Producer–Consumer
4. Prevent race conditions using semaphore variables

HARDWARE REQUIREMENTS

- PC or Laptop
- Minimum 2 GB RAM
(Anything less and you're basically coding on a toaster.)

SOFTWARE REQUIREMENTS

- Linux or Windows with C compiler
- GCC or Turbo C
- Command line / Terminal

THEORY

Semaphores are synchronization tools used to control access to shared resources in concurrent systems. They prevent race conditions by allowing only one process to access the critical section at a time.

There are two important semaphore concepts used here:

- **Binary semaphore (mutex):** Ensures mutual exclusion.
- **Counting semaphore:** Tracks the number of resources available.

In the Producer–Consumer problem:

- The **producer** adds data to a shared buffer.
- The **consumer** removes data from the shared buffer.
- A semaphore ensures both processes do not access the buffer at the same time.

The semaphore value acts as a signal:

- 1 means resource is free
- 0 means resource is occupied

This prevents both processes from entering the critical region simultaneously.

PROCEDURE

1. Write the C program containing producer and consumer functions.
2. Initialize semaphore variable to 1.
3. Allow the producer to add data to the buffer when semaphore is available.
4. Block consumer while producer is working.
5. After producing, release semaphore and signal the consumer.
6. Allow consumer to consume data and block producer.
7. Repeat the process until user stops the program.
8. Compile and run the program.

Commands:

```
gcc semaphore_pc.c -o semaphore_pc
./semaphore_pc
```

Source Code

```
#include<stdio.h>
#include<conio.h>
int n_semaphore=0;
int s_semaphore=1;
char s;

void producer()
{
    s_semaphore=0;
    if(!s_semaphore)
        printf("Now producer can add data to buffer\n");
    else
        printf("Critical Region \n");

    s_semaphore=1;
    signal_c();
}

void consumer()
{
    buffer_check();
    s_semaphore=0;

    if(!s_semaphore)
```

```

        printf("Consumer takes from the buffer\n");
    else
        printf("Critical Region \n");

    s_semaphore=1;
    signal_p();
}

signal_c()
{
    n_semaphore=n_semaphore+1;
    consumer();
    return 0;
}

signal_p()
{
    n_semaphore=n_semaphore-1;
    printf("Enter n to stop\n");
    scanf("%c",&s);
    if(s=='n')
        exit();
    return 0;
}

buffer_check()
{
    if(n_semaphore<=0)
    {
        printf("Buffer is empty\n");
        exit();
    }
    return 0;
}

void main()
{
    clrscr();
    n_semaphore=0;
    while(1)
    {
        producer();
    }
}

```

Sample Output

Now producer can add data to buffer
Consumer takes from the buffer
Enter n to stop
Now producer can add data to buffer
Consumer takes from the buffer
Enter n to stop
n

Exercise 2 – Semaphore Creation Function

To write a function that creates and initializes a semaphore set containing a single semaphore.

Example Code:

```
#include <sys/ipc.h>
#include <sys/sem.h>

int create_semaphore()
{
    key_t key = 1234;
    int semid;

    semid = semget(key, 1, 0666 | IPC_CREAT);

    semctl(semid, 0, SETVAL, 1);

    return semid;
}
```

PRACTICAL IMPORTANCE

- Used in operating systems to manage shared resources
- Prevents data inconsistency
- Essential in databases and real-time systems
- Foundation for multithreading and multiprocessing

MEASURING SATISFACTION

Students will be evaluated based on:

- Correct implementation of producer and consumer logic
- Proper use of semaphore variables
- Prevention of race conditions
- Correct program execution and output
- Understanding of synchronization concepts
- Performance during viva

CONCLUSION

The Producer–Consumer problem was successfully implemented using semaphores. The experiment demonstrated how mutual exclusion and synchronization prevent race conditions and ensure proper coordination between concurrent processes.

VIVA QUESTIONS

1. What is a semaphore?
2. Difference between binary and counting semaphore?
3. What is the Producer–Consumer problem?
4. What is a critical section?
5. What is mutual exclusion?
6. What causes race conditions?
7. How does a semaphore prevent data inconsistency?
8. What is busy waiting?

LAB EXPERIMENT 13-14

Connection Oriented Communication, Little and Big Endian

AIM

To understand and implement basic socket programming concepts by developing a client-server communication application in C.

OBJECTIVES

1. To understand the basics of Socket Programming.
2. To understand the working of Client–Server architecture.
3. To write and compile basic client and server programs.

LEARNING OUTCOMES

After this experiment, students will be able to:

1. Explain the working of sockets.
2. Implement basic TCP client-server programs.
3. Establish two-way communication between processes.

HARDWARE REQUIREMENTS

- PC / Laptop with minimum 2GB RAM
- Network interface (loopback is sufficient)

SOFTWARE REQUIREMENTS

- Linux Operating System
- GCC Compiler
- Terminal
- Telnet (for testing)

THEORY

A **socket** is one endpoint of a two-way communication channel between two programs running on a network. A client knows the hostname and port number of the server and tries to connect. If the server accepts the request, both client and server can communicate by reading from and writing to the socket.

The server:

- Creates socket
- Binds to IP and port
- Listens for connections
- Accepts clients

- Reads and writes data

The client:

- Creates socket
- Connects to server
- Sends and receives data

Source Code – Server

```
#include<stdio.h>
#include<string.h>
#include<sys/socket.h>
#include<arpa/inet.h>
#include<unistd.h>

int main(int argc , char *argv[])
{
    int socket_desc , client_sock , c , read_size;
    struct sockaddr_in server , client;
    char client_message[2000];

    socket_desc = socket(AF_INET , SOCK_STREAM , 0);
    if (socket_desc == -1)
    {
        printf("Could not create socket");
    }
    puts("Socket created");

    server.sin_family = AF_INET;
    server.sin_addr.s_addr = INADDR_ANY;
    server.sin_port = htons(8888);

    if(bind(socket_desc,(struct sockaddr *)&server , sizeof(server)) < 0)
    {
        perror("bind failed. Error");
        return 1;
    }
    puts("bind done");

    listen(socket_desc , 3);

    puts("Waiting for incoming connections...");
    c = sizeof(struct sockaddr_in);

    client_sock = accept(socket_desc, (struct sockaddr *)&client, (socklen_t*)&c);
    if (client_sock < 0)
    {
```

```

        perror("accept failed");
        return 1;
    }
    puts("Connection accepted");

    while((read_size = recv(client_sock , client_message , 2000 , 0)) > 0)
    {
        write(client_sock , client_message , strlen(client_message));
    }

    if(read_size == 0)
    {
        puts("Client disconnected");
    }
    else if(read_size == -1)
    {
        perror("recv failed");
    }

    return 0;
}

```

Source Code – Client

```

#include<stdio.h>
#include<string.h>
#include<sys/socket.h>
#include<arpa/inet.h>

int main(int argc , char *argv[])
{
    int sock;
    struct sockaddr_in server;
    char message[1000] , server_reply[2000];

    sock = socket(AF_INET , SOCK_STREAM , 0);
    if (sock == -1)
    {
        printf("Could not create socket");
    }
    puts("Socket created");

    server.sin_addr.s_addr = inet_addr("127.0.0.1");
    server.sin_family = AF_INET;
    server.sin_port = htons(8888);

```

```

if (connect(sock , (struct sockaddr *)&server , sizeof(server)) < 0)
{
    perror("connect failed. Error");
    return 1;
}

puts("Connected");

while(1)
{
    printf("Enter message : ");
    scanf("%s" , message);

    if(send(sock , message , strlen(message) , 0) < 0)
    {
        puts("Send failed");
        return 1;
    }

    if(recv(sock , server_reply , 2000 , 0) < 0)
    {
        puts("recv failed");
        break;
    }

    puts("Server reply :");
    puts(server_reply);
}

close(sock);
return 0;
}

```

PROCEDURE

1. Compile server:
2. gcc server.c -o server
3. Run server:
4. ./server
5. Compile client:
6. gcc client.c -o client
7. Run client:
8. ./client
9. Test using Telnet:
10. telnet localhost 8888

PRACTICAL IMPORTANCE

This experiment forms the foundation for network applications such as web servers, chat programs, and file transfer systems.

MEASURING SATISFACTION

- Successful compilation of client and server programs
- Correct creation of socket
- Proper binding to address and port
- Successful client-server connection
- Correct sending and receiving messages
- Clear understanding of socket functions during viva

CONCLUSION

Basic client-server communication using sockets was successfully implemented using TCP. The programs demonstrated how data can be exchanged between two processes.

VIVA QUESTIONS

1. What is a socket?
2. What is a port?
3. What is the difference between TCP and UDP?
4. Explain bind, listen, and accept.
5. What happens if the server is not running?

LAB EXPERIMENT 15

Advanced Client–Server Programs

AIM

To implement different client-server programs such as character increment, integer increment, and multi-threaded servers using socket programming.

OBJECTIVES

1. To send and receive characters between client and server.
2. To send and process integers.
3. To understand multi-threaded server programming.
4. To use the Make utility.

LEARNING OUTCOMES

Students will be able to:

1. Manipulate data on the server side.
2. Implement client-server logic for characters and integers.
3. Create multi-client servers.

THEORY

In advanced client-server communication:

- Client sends data.
- Server performs processing.
- Server sends back modified data.

Multi-threaded server creates a new thread for every client.

Exercise 2: Character Increment Programs

Server Logic:

- Read character from client.
- Increment character ($A \rightarrow B$).
- Send it back.

Client Logic:

- Send character.
- Receive updated character.

Exercise 3: Integer Increment Programs

Server Logic:

- Read integer from client.
- Increment value.
- Send it back.

Client Logic:

- Send integer.
- Receive updated integer.

Exercise 4: Multi-threaded Server

Server Features:

- Uses pthread library.
- Handles multiple clients simultaneously.
- Uses Makefile for compilation.

PROCEDURE

1. Write client and server programs.
2. Create Makefile.
3. Compile using:
4. `make`
5. Run server then connect multiple clients.

PRACTICAL IMPORTANCE

These experiments simulate real-world server behavior used in:

- Banking systems
- Chat servers
- Online gaming

MEASURING SATISFACTION

- Correct implementation of character increment program
- Correct implementation of integer increment program
- Successful handling of multiple clients
- Proper use of threads in server program
- Correct usage of Makefile and compilation using `make`

- Correct program output
- Conceptual understanding during viva

CONCLUSION

Client-server communication was successfully implemented for character and integer manipulation. Multi-threading allowed simultaneous client handling.

VIVA QUESTIONS

1. What is multithreading?
2. Difference between process and thread?
3. What is `pthread_create()`?
4. Why use Makefile?
5. What is concurrency?

Lab Experiment 16 & 17

Messages and Queues

AIM

To study and implement inter-process communication using System V Message Queues in Linux by creating producer and consumer processes.

OBJECTIVES

1. To understand the concept of Message Queues in inter-process communication (IPC).
2. To understand how processes communicate using message queues.
3. To implement message queue creation, sending, and receiving operations.
4. To observe queue status using message control operations.

LEARNING OUTCOMES

After completing this experiment, students will be able to:

1. Explain the working of System V Message Queues.
2. Use msgget(), msgsnd(), msgrcv(), and msgctl() system calls.
3. Implement producer-consumer problems using message queues.
4. Analyze process synchronization using message queues.

HARDWARE REQUIREMENTS

- A computer/laptop with at least:
 - 2 GB RAM (anything less and Linux will cry quietly)
 - x86/x64 processor

SOFTWARE REQUIREMENTS

- Linux Operating System (Ubuntu, Debian, Fedora, etc.)
- GCC compiler
- Terminal / Command Line Interface

THEORY

Message Queues are a form of Inter-Process Communication (IPC) that allow multiple processes to exchange data asynchronously.

In Linux, System V Message Queues are maintained by the kernel in a data structure called msqid_ds.

Each message queue:

- Has permissions stored in ipc_perm.
- Stores messages in FIFO order.
- Maintains two wait queues:
 - One for sending processes.
 - One for receiving processes.

Each message consists of:

- A mandatory long message type.

- A data structure containing text or other information.

Key system calls used:

- `msgget()` → creates or accesses a message queue.
- `msgsnd()` → sends a message.
- `msgrcv()` → receives a message.
- `msgctl()` → controls the message queue.

If the queue is full, the sender process is blocked.

If no matching message exists, the receiver process is blocked.

Yes, the kernel babysits both.

PROCEDURE

1. Open the Linux terminal.
2. Create and save a producer program file (e.g., `producer.c`).
3. Create and save a consumer program file (e.g., `consumer.c`).
4. Compile both programs using:
5. `gcc producer.c -o producer`
6. `gcc consumer.c -o consumer`
7. Run the producer program with a message type:
8. `./producer 5`
9. Run the consumer program with the same message type:
10. `./consumer 5`
11. Observe:
 - Message sent by producer
 - Message received by consumer
 - Queue statistics displayed using `msgctl()`

PRACTICAL IMPORTANCE

This experiment helps students:

- Understand real-world IPC used in operating systems.
- Learn synchronization between processes.
- Understand how operating systems manage shared resources.
- Apply concepts in server–client and distributed systems.

Basically, it teaches how processes talk instead of fighting silently.

MEASURING SATISFACTION

Student performance can be evaluated based on:

1. Successful creation of message queues.
2. Correct sending and receiving of messages.
3. Proper handling of blocking and non-blocking modes.
4. Understanding of queue statistics.
5. Correct explanation during viva.

CONCLUSION

In this experiment, System V Message Queues were successfully used to exchange messages between two processes. The producer–consumer problem was implemented, demonstrating process synchronization, message typing, and kernel-managed communication mechanisms. The experiment showed how Linux efficiently handles inter-process communication using message queues.

VIVA QUESTIONS

1. What is Inter-Process Communication (IPC)?
2. What is a Message Queue?
3. Difference between Message Queue and Shared Memory.
4. Explain the function of `msgget()`.
5. What is the purpose of `msgsnd()`?
6. How does `msgrcv()` work?
7. What is the role of `msgctl()`?
8. What happens when a message queue becomes full?
9. Explain blocking vs non-blocking mode.
10. What is the purpose of the message type?